VOLT: Visual Output for Layered Topologies

Version 1.0 User Guide

Fabrizio De Caro, and Alfredo Vaccaro

I. Motivations

Visualizing power system simulation results directly within MATLAB remains a significant challenge. Existing tools present critical functional limitations:

- **MATPOWER Built-in Utilities:** Only scale branch line widths according to nominal voltage ratings, lacking the ability to project calculated simulation metrics (e.g., active/reactive power flows, line loadings, or voltage profiles) onto the network topology [1].
- **Static Graph Tools (e.g., GridVis):** Construct network graphs from static spreadsheet data without spatial layout optimization, suffer from directionally blind flow representations, and lack parametric customization [2].

To bridge this gap, **VOLT** (*Visual Output for Layered Topologies*) introduces a flexible, open-source MATLAB library designed to overlay multi-layered simulation results—derived directly from MATPOWER datasets—onto dense networks.

II. Core Contributions:

- **Dynamic Flow Rendering:** Automatically reorients edge arrows according to physical active power flow directions and dynamically scales colormaps.
- **Multi-Layered Architecture:** Separates node metrics, branch metrics, and visual legends into independent layers to avoid graphic overlap.
- **Publication-Ready Outputs:** Generates fully customizable MATLAB figure files (`.fig`) and vector formats (`.pdf`/`.eps`) optimized for academic papers and teaching.

III. Scope and Limitations

This software is primarily intended for education and research applications, focusing on the visual analysis of power grids with multiple voltage levels and up to 2,500 buses.

The visualization algorithm aims to maximize the clarity of the grid topology rather than strictly reproducing physical or geographic element distributions. The chosen layout engine generates structured, regular geometric node arrangements. Nevertheless, custom spatial coordinates can be enforced when required, as illustrated in this guide.

While computational execution times are analyzed and optimized as far as practical, achieving absolute algorithmic speed is intentionally secondary during development. The

F. De Caro and A. Vaccaro are with the Power System Research Group, Department of Engineering, University of Sannio, Benevento 82100, Italy (e-mail: fdecaro@unisannio.it; vaccaro@unisannio.it).

For any issues or questions regarding the software implementation, please contact fdecaro@unisannio.it.

main objective is to establish a solid functional baseline to stimulate the research community toward developing advanced, time-efficient graphical tools in MATLAB. Despite this design trade-off, the tool exhibits high responsiveness within its target operating range ($\sim$1,000 buses), achieves reasonable execution times up to 2,500 buses, and guarantees smooth GUI navigation once figures are fully rendered. To ensure optimal rendering performance and seamless user interaction—especially when processing medium-to-large networks—a dedicated graphics card is strongly recommended.

IV. Library Contents

In Version 1.0, the **VOLT** library is delivered as a lightweight, standalone MATLAB tool consisting of a single core function file:

- `VOLT.m`: Contains the entire rendering architecture, data preprocessing, hash-mapping algorithms, and visual customization options.

V. How to Cite

A companion paper detailing the architectural framework and applications of `VOLT` is currently under peer review. In the meantime, if you use this library in your research, please cite this software repository and technical documentation:

```
@misc{VOLT2026,
  author       = {De Caro, Fabrizio and Vaccaro, Alfredo},
  title        = {{VOLT}: Visual Output for Layered Topologies (Version
    1.0)},
  year         = {2026},
  howpublished = {MATLAB library, University of Sannio},
  note         = {Available at GitHub repository},
  url          = {https://github.com/username/VOLT}
}
```

Listing 1. BibTeX entry for citing VOLT.

Note: This section will be updated with the full journal paper reference upon acceptance.

VI. Requirements

- Requires MATLAB (R2023b or later) and Graph Theory library routines.
- The MATPOWER package is required for data structure compatibility and for the `ext2int` function.

VII. Installation & Setup

VOLT does not require a complex installation process or external compilation. Users can integrate it into their MATLAB environment using one of two methods:

A. Method 1: Direct Working Directory (Quick Setup)

Copy the `VOLT.m` file directly into the working directory where your main MATLAB script or power flow analysis resides.

B. Method 2: Global MATLAB Path Addition (Recommended)

To avoid maintaining multiple copies of `VOLT.m` across different project folders, place the file in a permanent folder and add it to the MATLAB search path. This can be done via the Command Window or the Graphical User Interface (GUI):

1) **Via MATLAB Command Window:** Execute the following commands to add the folder to the active session and save it permanently for future sessions:

```
addpath('C:/Path/To/Your/VOLT_Folder'); % Replace with your actual
    path
savepath;                               % Saves path for future
    sessions
```

2) **Via MATLAB GUI Environment:**
   - Open MATLAB and navigate to the **Home** tab in the top toolbar.
   - Click on the **Set Path** icon (within the *Environment* section).
   - In the dialog box, click **Add Folder...** and select the directory containing `VOLT.m`.
   - Click **Save** to retain the configuration, then click **Close**.

Once added to the path, `VOLT` can be invoked from any working directory across your MATLAB environment.

VIII. Quick Start

To quickly render a network, load a standard MATPOWER case (version 2.0 format), execute a power flow analysis (such as AC, DC, or OPF), and pass the solution structure directly to `VOLT`:

```
% 1. Load standard MATPOWER benchmark system
mpc = loadcase('case5');

% 2. Execute power flow analysis
results = runpf(mpc);

% 3. Render network with default settings
VOLT(results);
```

Listing 2. Quick Start Example.

The function signature accepts six input arguments, where only the first (`results`) is mandatory. The remaining parameters automatically revert to their default values if omitted:

```
% Function Signature

VOLT(results, branch_var, terminal, bus_var, show_labels, opts)
```

Note: For a detailed description of each input argument and its default configuration, see Section IX (Table I).

IX. VOLT Architecture

This section outlines the main features and architecture of the VOLT library:

A. Input Data

VOLT is implemented as a standalone MATLAB function that processes the MATPOWER output case in the standard case format and computes user-defined bus and branch metrics.

- **Network Data & Results**: Grid parameters and power flow solutions structured according to the MATPOWER version 2 format;
- **Branch Metrics**: The specific quantities to visualize along each network branch.
- **Terminal Reference**: The computational perspective for lumped branch models (i.e., sending/arriving end);
- **Node Metrics**: The specific variables to display at each bus/node;
- **Display Labels**: A binary control variable to enable or disable the rendering of any label;
- **Customization Options** (Optional): A dedicated data structure containing layout preferences, topology configurations, and legend styling.

Table I lists the function input arguments and computational options, whereas Table II details the rendering layout options.

TABLE I
INPUT ARGUMENTS

Argument	Data Type	Options
`results`	Struct	
`branch_var`	String	`'P'` (Active Power), `'Q'` (Reactive Power), `'S'` (Apparent Power), `'I'` (Current Magnitude), `'L'` (Power Loading)
`terminal`	String	`'F'` (From), `'T'` (To)
`bus_var`	String	`'VM'` (Voltage Magnitude), `'VA'` (Voltage Phase)
`show_labels`	String	`'Y'` (Yes), `'N'` (No)
`opts`	Struct	Custom fields (optional)

B. Data Verification and Preprocessing

VOLT validates input data format integrity before rendering. It checks the dimensional coherence of the MATPOWER `results` structure—obtained by executing `runpf` on a target network case `mpc`—and verifies the presence of active and reactive power flow data at both sending and receiving branch terminals. When available, the tool automatically computes the corresponding apparent power and current magnitudes. If branch loading visualization is requested, VOLT checks for non-zero branch power ratings (expressed in MVA under `RateA`/`B`/`C`). If these rating data are missing or set to zero, the tool automatically falls back to rendering apparent power flows to ensure visual consistency.

C. Graph Construction and Data Mapping

To generate the network graph, VOLT extracts the user-selected bus and branch metrics to populate the nodes and edges of a directed graph. The MATPOWER utility `ext2int` is applied to the `'results'` structure to convert non-contiguous external bus IDs into contiguous internal indices if they are not already ordered. This operation ensures full interoperability when processing results obtained from other solvers formatted according to the MATPOWER case standard.

To optimize visualization clarity, users can isolate specific sub-networks using the voltage filtering structure (`vm_range`). When active, network components—including

TABLE II
OPTIONS PARAMETERS STRUCTURE (opts) [PART 1]

Field Name	Default	Description
Color & Rendering Options		
vm_breaks	[0.90,0.95, 1.00,1.05, 1.10]	Reference thresholds for bus voltage magnitude colormaps and colorbar tick labels
va_breaks	[-30,-15,0 15,30]	Reference thresholds for bus voltage phase colormaps and colorbar tick labels
br_breaks	[]	Reference thresholds for branch variable colormaps and colorbar tick labels
branch_w	[0.8,3.6]	Min and max branch line widths
vm_range	[]	Lower and Upper voltage values for element filtering
filter_mode	'strict'	Voltage filtering logic: 'strict' excludes edges connecting buses with differing rated voltages; 'base' includes them
coord_regen	0	Forces regeneration of graph node coordinates (1 = active)
br_rate	'A'	Branch rating limit for loading: A, B, or C
Geometry, Font & Window Layout		
arrowsize	10	Branch flow directional arrow size
markersize	6	Base node marker size
fontsize	9	General font size for labels and legends
layoutgraph	'force'	MATLAB digraph positioning algorithm
graphit	5000	Maximum iterations for graph positioning
		Continue on Table III→

buses, branches, and generators—are filtered from the respective results fields based on their terminal rated voltages. The inclusion of boundary branches connecting differing voltage levels (i.e., transformers) is governed by the selected filtering logic ('strict' versus 'base' mode). Following sub-network extraction, ext2int is re-executed to re-establish contiguous indexing. To maintain traceability, the pre-filtering bus IDs are preserved in an auxiliary column of the 'bus' field, enabling optional dual-ID rendering via 'auxbusID_on'. Subsequently, de-energized elements are removed according to their operational status in the branch table.

The core topology is built using MATLAB's digraph engine, initialized by mapping the grid's bus and branch data structures to the graph's nodes and edges, respectively. By default, digraph assigns edge orientations based on the node order in the input connection matrix (pointing from the first column to the second). Consequently, to reflect physical power flow, the node order of each branch is dynamically reversed according to the calculated flow direction.

Notably, MATLAB's digraph constructor internally reorders edge indices based on node connectivity. As a result, the graph's internal edge sequence diverges from the original branch order in the source data matrix, often swapping terminal nodes. Reorienting edge directions to match active power flow further exacerbates this discrepancy. To prevent indexing mismatches and ensure correct attribute assignment, VOLT establishes an explicit mapping vector between MATPOWER branches and graph edges.

To preserve data alignment, VOLT implements a dedicated pairing algorithm that maps the original MATPOWER branch parameters onto the reordered graph structure via hash-based lookup, achieving constant-time $\mathcal{O}(1)$ query complexity. For each branch in the network, the algorithm generates a primary key from the ordered tuple of terminal buses (f_{bus}, t_{bus}), along with a reversed key (t_{bus}, f_{bus}) to account for potential direction

TABLE III
OPTIONS PARAMETERS STRUCTURE (opts) [PART 2]

Field Name	Default	Description
→ From Table II		
Geometry, Font & Window Layout		
figpos	[100,100,] 1250,800]	Figure window position and dimensions
axpos	[0.05,0.1, 0.85,0.85]	Plot axes area margins within the figure
Visibility Flags & Spatial Offsets		
bus_lab_on	1	Enables node data labels (0 disables)
edge_lab_on	1	Enables edge data labels (0 disables)
trs_lab_on	1	Enables transformer labels (0 disables)
auxbusID_on	0	Disables auxiliary bus ID (1 enables)
bus_dl	0	Node detail level: 0 (ID), 1 (ID, VM, VA), or 2 (Full details)
edge_dl	1	Branch detail level: 0 (P,Q), 1 (P,Q,S), or 2 (P,Q,R,C)
trs_dl	0	Transformer detail level: 0 (label), or 1 (label, tap ratio, shift angle)
branch_sign	'relative'	Changes P and Q branch label sign according to the edge direction. 'tabular' shows sign aligned with the tabular output
leg_trs_on	1	Enables transformer legend (0 disables)
leg_note_on	1	Enables note legend (0 disables)
Figure Export Options		
savefig	0	1 enables figure saving (.fig and pdf/eps)
figname	[]	Default name with timestamp if empty
figformat	'.pdf'	Format for fast exportation

flips induced by power flow. By querying both key sets against the graph's internal edge list, the algorithm unambiguously retrieves the matching branch indices. This yields a deterministic mapping vector that translates the raw MATPOWER ordering into the internal digraph sequence.

VOLT supports flexible network layouts via custom spatial coordinates, user-defined positioning options by inserting an optional Coord field directly into the results structure, or automated layout generation. When coordinates are omitted from the results structure, VOLT defaults to a force-directed layout 'force' with 'UseGravity', true). This provides an optimal spatial arrangement that avoids node and edge overlap while minimizing computational overhead.

To enhance visual perception across heterogeneous data types, VOLT dynamically adapts color schemes based on the displayed physical metric. Magnitude distributions and bidirectional operational variables utilize distinct, continuous colormaps tailored to highlight system gradients and critical operating thresholds.

To visually represent grid levels, edge line widths are scaled according to the rated voltage. For edges connecting nodes whose corresponding buses are at different voltage levels via transformers, the reference voltage corresponds to the bus with the smaller index.

Node colors indicate voltage magnitude ('VM') or phase angle ('VA') variations. By default, per-unit voltage magnitudes (0.90–1.10 p.u.) transition smoothly from blue (low) through green (nominal) to red (high), with custom thresholds dynamically interpolated across the chosen colormap via MATLAB's interp1. To distinguish bus types, the nodes are rendered using distinct geometric markers.

Node labels display detailed bus-level operational metrics, including the bus ID, voltage magnitude, and phase angle relative to the slack bus. Furthermore, they show the net active and reactive injected power, alongside their disaggregation into generation, load, and shunt components, as well as the number of online generators connected to each bus.

Branch labels display active power, reactive power, or current magnitude calculated at the selected terminal, alongside off-nominal tap ratios and phase shifts for transformer branches. To maintain mathematical consistency with graph orientations, VOLT supports two sign conventions via `'branch_sign'`: raw values referenced to their alignment with the tabular terminal order, or a relative mode where label signs automatically align with the calculated flow arrow ($+$ for forward, $-$ for reverse).

D. Rendering Steps

To overcome native plotting limitations in MATLAB's standard `graph` routines, VOLT overlays distinct graph layers:

1) **Base Layer**: Renders the directed graph topology and branch styling.
2) **Node Geometry Layer**: A scatter plot overlay that formats bus shapes according to their functional type ($\square, \bigcirc, \triangle$).
3) **Edge Metric Colorbar**: Displays the color scale for edge variables on the left margin.
4) **Node Metric Colorbar**: Displays the color scale for node variables on the right margin.
5) **Node Labels**: Displays textual information about the bus.
6) **Edge Labels**: Displays textual information about the branch (Active/Reactive/Apparent Power, Current).
7) **Transformer Labels**: Displays if the edge represents a transformer and details about the tap ratio and shift.
8) **Node Legend**: Displays the legend for node symbols on the upper left margin.
9) **Edge Width Legend**: Displays the edge width and the corresponding voltage rate value.
10) **Edge Legend**: Displays the legend for branch edge interpretation on the lower right margin.
11) **Figure Export**: It produces a MATLAB `'.fig'` for post-editing, and a graphic format suitable for fast exportation in LaTeX.

This multi-layered approach enables independent visual customization for distinct grid components and legends without visual overlap.

E. Hierarchy Between Label Visualization Options

To clarify the control logic governing text overlays, the label rendering pipeline follows a strict three-tier hierarchy:

1) **Master Switch (`show_labels`):** Acts as the top-level control flag (`'Y'` or `'N'`). If set to `'N'`, all text overlays are globally suppressed regardless of individual sub-options.
2) **Category Visibility Flags (`*_lab_on`):** When the master switch is active (`'Y'`), individual visual categories are toggled via `bus_lab_on`, `edge_lab_on`, and `trs_lab_on` (1 to enable, 0 to disable).

3) **Detail Level Granularity (`*_dl`):** Controls the density of displayed physical variables for each enabled category, ranging from minimal identifiers to complete power flow metrics (`bus_dl`, `edge_dl`, `trs_dl`).

The structural relationship between these options is summarized in the visual hierarchy below:

Level 1: Master Switch	**Level 2: Category Flags (`opts`)**	**Level 3: Detail Level (`opts`)**
`show_labels = 'N'` → Disables all labels		
`show_labels = 'Y'` → Enters hierarchy →	`bus_lab_on = 1` →	`bus_dl` (0: ID, 1: Basic, 2: Full details)
	`edge_lab_on = 1` →	`edge_dl` (0: P,Q, 1: P,Q,S, 2: Full)
	`trs_lab_on = 1` →	`trs_dl` (0: Label, 1: Tap ratio & shift)

F. Output Data

VOLT does not return variables to the main workspace; instead, it opens a MATLAB Figure object. Additionally, if enabled in the options, it saves a copy for subsequent manual edits along with an express export in the desired format for inclusion in documents.

X. Examples

This section evaluates several benchmark test cases to demonstrate the visualization capabilities and core features of VOLT.

A. 5-bus System

The first case study considers the standard 5-bus system, where Fig. 1 shows the native MATPOWER plot as a reference.

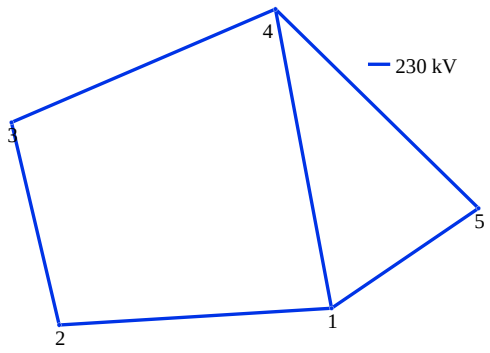


Fig. 1. 5-bus System render by MATLAB `plot_mpc`.

Listing 3 provides the MATLAB script used to generate Fig. 2. The power flow results in Table IV validate the consistency between the plotted edge labels and arrow directions. For instance, edge 3 corresponds to the branch connecting buses 1 and 5. According to Table IV, active power flows from bus 1 to bus 5. As rendered by VOLT, when adopting the sending terminal (`'F'`) reference, the active power injection is negative (-226.3 MW), and the edge arrow is dynamically oriented from node 5 to node 1 to reflect the physical direction of power flow.

```m
% 1. Load standard MATPOWER benchmark system
mpc = loadcase('case5');
% 2. Execute power flow analysis
results = runpf(mpc);
% 3. Call VOLT
opts.fontsize   = 20;
opts.markersize = 12;
opts.arrowsize  = 20;
opts.branch_sign = 'tabular';
VOLT(results, 'S', 'F', 'VM', 'Y',opts);
```

Listing 3. VOLT configuration and execution code for Fig. 2.

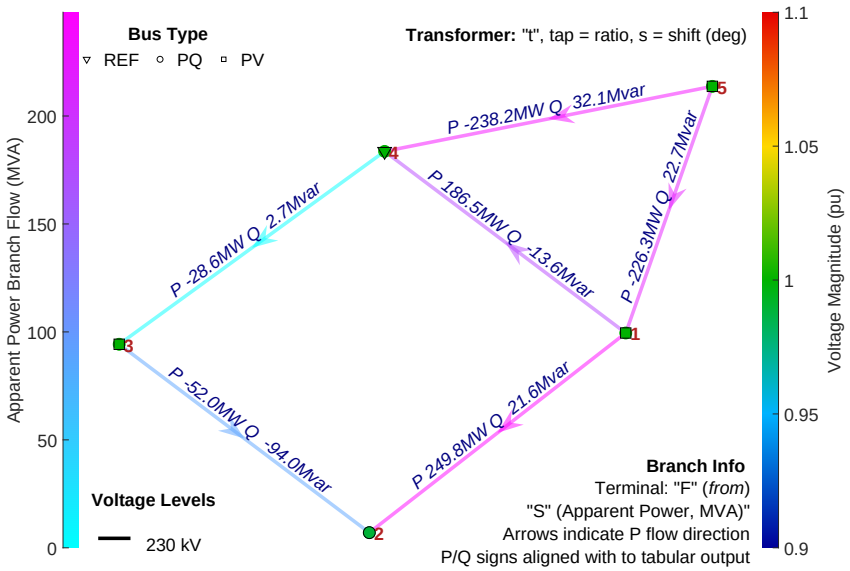


Fig. 2. 5-bus System render by VOLT.

TABLE IV
CASE 5 POWER FLOW SOLUTION AND NETWORK SUMMARY

(a) Branch Power Flow Results

Branch	From	To	From Bus Injection		—
#	Bus	Bus	P (MW)	Q (MVAr)	
1	1	2	249.77	21.60	
2	1	4	186.50	-13.61	
3	1	5	-226.27	22.74	
4	2	3	-51.99	-93.97	
5	3	4	-28.63	2.65	
6	4	5	-238.19	32.15	

(b) Bus Results and Power Flow Summary

Bus	Voltage		Generation		Load	
#	Mag (p.u.)	Ang (deg)	P (MW)	Q (MVAr)	P (MW)	Q (MVAr)
1	1.000	3.273	210.00	30.73	—	—
4	1.000	0.000*	5.03	184.12	400.00	131.47
5	1.000	4.112	466.51	-38.21	—	—

If `opts.bus_dl` is set to `2`, all bus details are visualized for each bus. This is useful for visual inspection. However, for a limited number of buses and when adjusting the

font sizes is possible, generating printable figures as shown in Fig. 3.

Table IV shows the consistency between visualized and tabular quantities for buses 1, 4, and 5. The markers correctly identify bus 4 as the slack bus. In addition, the detailed labels return the bus area and the number of active generators for each node.

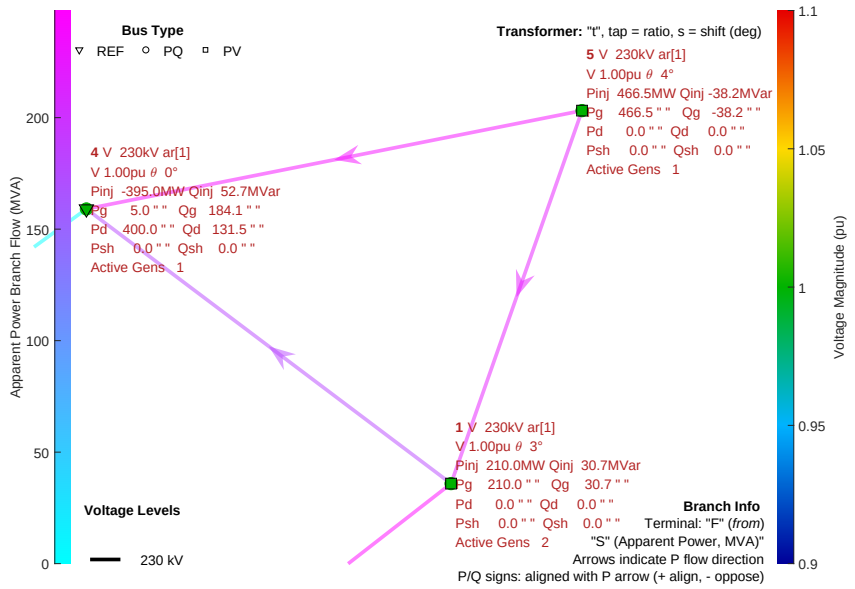


Fig. 3. 5-bus System with the bus highest level of details.

B. IEEE 39-bus System

The IEEE 39-bus system serves as a benchmark for visualizing branch loading percentages based on line power ratings (`Rate A`), as shown in Fig. 4. In this scenario, line (4)-(14) is disconnected, inducing an overload on line (6)-(11). To visually distinguish relative metrics (`'L'`) from absolute quantities, the `jet` colormap is applied. Listing 4 demonstrates preserving the original network topology via pre-initialized node coordinates when branches are deactivated.

```
% 1. Load standard MATPOWER benchmark system
mpc = loadcase('case39');
% 2. Extract initial node layout to preserve topology across grid
    filtering
h=plot(graph(mpc.branch(:,1),mpc.branch(:,2)),'Layout', opts.layoutgraph,
    'Iterations', opts.graphit, 'UseGravity', true);
mpc.Coord = [h.XData',h.YData'];
% 3. Deactivate branch and execute power flow analysis
mpc.branch(9,11) = 0;
opts.arrowsize   = 15;
opts.markersize  = 9;
opts.fontsize    = 13;
results = runpf(mpc);
% 4. Call VOLT
VOLT(results, 'L', 'F', 'VM', 'Y',opts);
```

Listing 4. VOLT configuration and execution code for Fig. 4.

C. IEEE 118-bus System

This system features a two-voltage-level grid. Hence, VOLT automatically detects the different voltage levels and renders them with distinct edge widths, adding dedicated

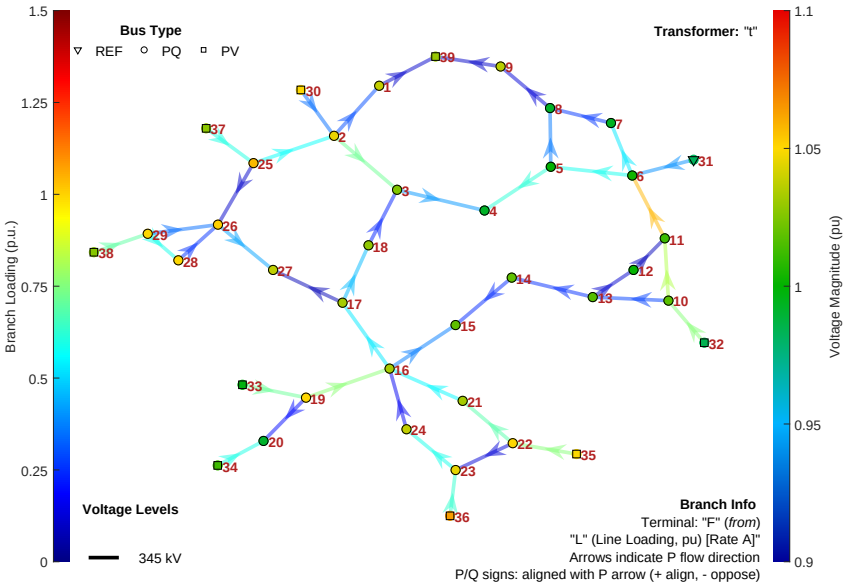


Fig. 4. IEEE 39-bus System.

labels to transformer branches, as shown in Fig 5. Under this configuration, transformer details are displayed at the lowest level of detail.

In a multivoltage grid, `opts.vm_range` visualizes only specific nominal voltage levels. This field requires a two-element array in kV, specifying the lower and upper bounds for filtering. An internal algorithm identifies all grid elements connected to the retained buses and automatically removes those linked to excluded buses. Fig. 6 illustrates the filtered grid topology of the IEEE 118-bus system.

To ensure traceability between the new and pre-filtering bus ID enumerations, the tool automatically activates the auxiliary bus visualization feature (`opts.auxbusID_on = 1`), mapping original bus indices alongside internal renumbered node labels. Listing 5 shows the code to filter elements according to the voltage rate value and to preserve coordinates as shown in the 39-bus case.

```
% 1. Load standard MATPOWER benchmark system
mpc = loadcase('case118');
% 2. Extract initial node layout to preserve topology across grid
    filtering
h=plot(graph(mpc.branch(:,1),mpc.branch(:,2)), 'Layout', opts.layoutgraph,
     'Iterations', opts.graphit, 'UseGravity', true);
mpc.Coord = [h.XData',h.YData'];
% 3. Deactivate branch and execute power flow
results = runpf(mpc);
% 4. set filter and Call VOLT
opts.vm_range = [344,346] %kV
VOLT(results, 'S', 'F', 'VM', 'Y',opts);
```

Listing 5. VOLT configuration and execution code for Fig. 6.

D. PEGASE 1354-bus System

In this final case, a large grid rendering illustrates VOLT's capability to rapidly generate a well-distributed map that clearly highlights the plot variable's magnitude, despite the high component count and the minimal style applied to reduce visual clutter. For instance, the 380 kV grid portion and surrounding 220 kV subnetworks are easily identifiable.

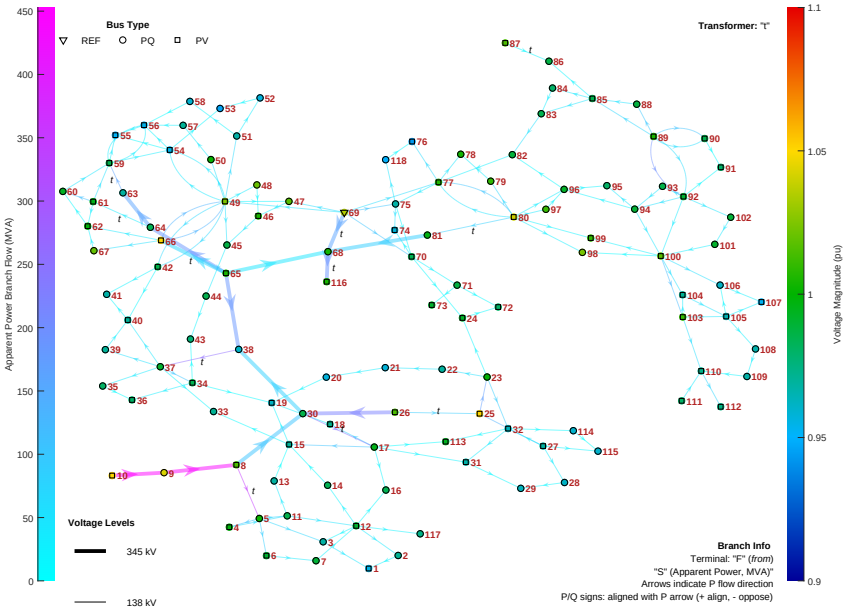


Fig. 5. IEEE 118-bus System (a): Full network.

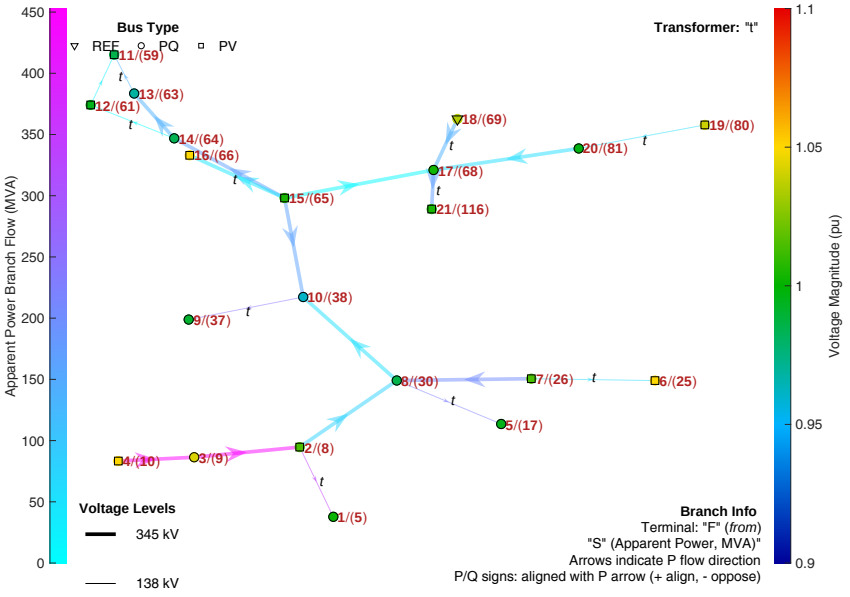


Fig. 6. IEEE 118-bus System (b): 345kV grid portion.

Furthermore, the range of voltage magnitude is effectively displayed, utilizing a distinct color scale from that used for the branches. The full layout generation and rendering pipeline for the 1354-bus system requires less than 50 seconds on a workstation equipped with Intel i7 9700 3.0GHz, 16 GB RAM DDR4 and NVIDIA GeForce RTX 2060.

E. Layout Generation and Rendering Test

This section presents the computational benchmark evaluated on the MATPOWER 8.0 test library. Out of 77 available cases in the `data` directory, 55 cases with up to 2,500 buses are selected. Larger cases are omitted at this stage, as the current release targets core rendering capabilities, whereas scaling to tens of thousands of buses is reserved for future work. For each case, execution is iterated 50 times to compute average execution

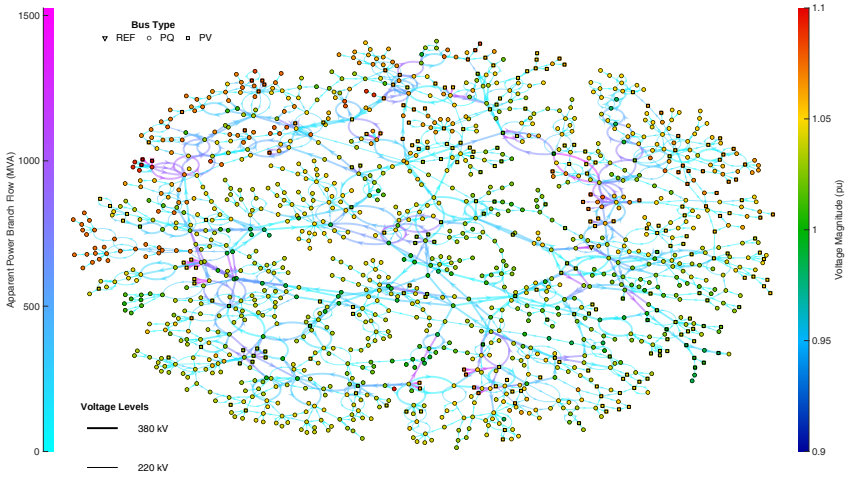
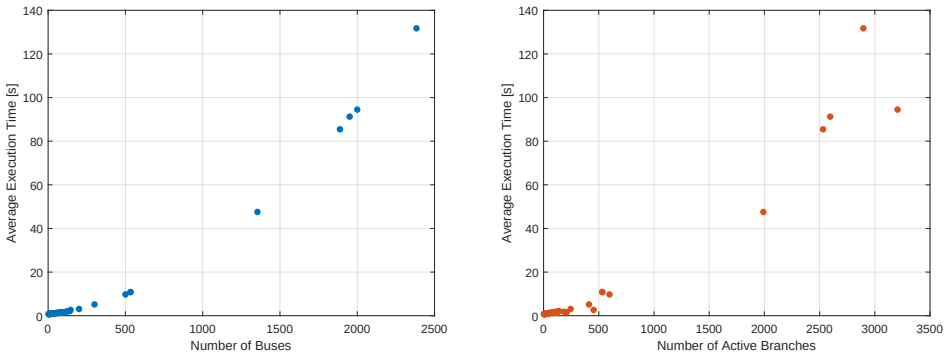


Fig. 7. PEGASE 1354-bus System.

times. All simulations are conducted with label rendering turned off and quick start factory settings.

Table V presents the benchmark metrics for each test case, reporting the mean execution time and standard deviation. Additionally, scatter plots in Figs. 8a and 8b illustrate software performance as a function of the number of buses and active branches.

As observed, the computational execution time increases quadratically with the number of buses and branches. This trend stems from adopting a force-directed layout algorithm to distribute nodes across the space, which exhibits a computational complexity of $\mathcal{O}(N^2)$ due to evaluating attractive and repulsive forces between all node pairs at each iteration. Nevertheless, employing $\mathcal{O}(1)$ hash-map data structures prevents data retrieval overhead from further exacerbating the overall complexity, ensuring that execution times remain strictly governed by the spatial force calculations.



(a) Number of Buses vs Computational Time. (b) Number of Branches vs Computational Time.

Fig. 8. Computational time analysis as a function of network scale.

TABLE V
MATPOWER CASES, LAYOUT GENERATION AND GRAPH RENDERING TIME.

Nr. Case	Analyzed System	Bus Number	Branch Number	Average Time [s]	Std Dev [s]
1	case10ba	10	9	0.74	0.06
2	case118	118	186	1.80	0.03
3	case118zh	118	117	1.92	0.07
4	case12da	12	11	0.83	0.01
5	case1354pegase	1354	1991	47.58	0.31
6	case136ma	136	135	2.09	0.05
7	case14	14	20	0.95	0.03
8	case141	141	140	2.16	0.01
9	case145	145	453	2.65	0.07
10	case15da	15	14	0.86	0.02
11	case15nbr	15	14	0.85	0.01
12	case16am	15	14	0.86	0.03
13	case16ci	16	13	0.87	0.03
14	case17me	17	16	0.94	0.01
15	case18	18	17	0.88	0.01
16	case1888rte	1888	2531	85.44	0.54
17	case18nbr	18	17	0.83	0.05
18	case1951rte	1951	2596	91.25	0.85
19	case22	22	21	0.92	0.03
20	case2383wp	2383	2896	131.73	0.37
21	case24_ieee_rts	24	38	0.94	0.03
22	case28da	28	27	0.94	0.05
23	case30	30	41	0.98	0.02
24	case300	300	411	5.16	0.05
25	case30Q	30	41	0.99	0.02
26	case30pwl	30	41	0.98	0.07
27	case33bw	33	32	1.01	0.02
28	case33mg	33	32	1.01	0.01
29	case34sa	34	33	1.01	0.01
30	case38si	38	37	1.04	0.05
31	case39	39	46	1.08	0.02
32	case4_dist	4	3	0.76	0.01
33	case4gs	4	4	0.75	0.06
34	case5	5	6	0.76	0.02
35	case51ga	51	50	1.25	0.01
36	case51he	51	50	1.15	0.05
37	case533mt_hi	533	532	10.81	0.04
38	case533mt_lo	533	532	10.84	0.10
39	case57	57	80	1.26	0.03
40	case60nordic	60	88	1.24	0.05
41	case69	69	68	1.33	0.02
42	case6ww	6	11	0.85	0.01
43	case70da	70	68	1.35	0.01
44	case74ds	74	73	1.48	0.01
45	case85	85	84	1.59	0.01
46	case89pegase	89	210	1.58	0.01
47	case9	9	9	0.79	0.01
48	case94pi	94	93	1.65	0.08
49	case9Q	9	9	0.80	0.04
50	case9target	9	9	0.79	0.01
51	case_ACTIVSg200	200	245	3.08	0.01
52	case_ACTIVSg2000	2000	3206	94.47	0.46
53	case_ACTIVSg500	500	597	9.74	0.06
54	case_RTS_GMLC	73	120	1.29	0.01
55	case_ieee30	30	41	0.92	0.06

XI. License

Copyright (c) 2026 Fabrizio De Caro, Alfredo Vaccaro / Power Systems Research Group / University of Sannio (Università degli Studi del Sannio), Benevento, Italy.

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see https://www.gnu.org/licenses/.

References

[1] R. D. Zimmerman, C. E. Murillo-Sánchez, and R. J. Thomas, "Matpower: Steady-state operations, planning, and analysis tools for power systems research and education," *IEEE Transactions on Power Systems*, vol. 26, no. 1, pp. 12–19, 2011.

[2] H. Mirtaheri, "Grid-visualization-in-Matlab: Power System Visualization for MATLAB-Based Analysis," https://github.com/mirtaheri/Grid-visualization-in-Matlab, 2017, gitHub repository, accessed: 2026-08-31.